

2026《Linux内核原理与分析》第六周作业：追踪 write 系统调用从 system_call 到 iret

姓名：叶丁再

学号：20262809

实验主题：分析 32 位 Linux 3.18.6 的 system_call 中断处理过程

一、实验目标与环境

本实验沿用上周选择的 write 系统调用。上周的实验在 32 位 x86 ABI 下通过 libc API 和内嵌汇编使用 write；本周进一步观察它在 Linux 内核中的处理函数，并结合 Linux 3.18.6 的 x86 入口汇编，分析从 system_call 到 iret 的主要路径，尤其关注返回用户态前可能发生的调度。

实验使用实验楼提供的 Linux 虚拟机、Linux 3.18.6 内核及 QEMU/GDB。GDB 在内核的 SyS_write 处理函数处命中，源码位置为 fs/read_write.c:577。对应的参数显示 fd=1、count=18；同时 QEMU 窗口显示 MenuOS 提示符和输入命令的上下文。

二、GDB 跟踪 write 内核处理函数

在 QEMU 中运行 MenuOS 时，GDB 断点停在 SyS_write。info args 显示：

- fd = 1：标准输出；
- buf = -1217257472：GDB 以有符号十进制形式显示的缓冲区参数；
- count = 18：本次写入的字节数。

这里的负数显示不能据此解释为无效指针或错误返回；它是调试器显示指针参数时的数值形式。实验截图没有读取缓冲区内容，因此本文不推断这 18 个字节的具体文本。

The screenshot displays a web browser window on the left with the title "分析 system_call 中断处理过程". The page content includes a list of bullet points detailing the experiment's goals and environment, and a section for submitting the blog post URL. On the right, a GDB terminal window is open, showing a breakpoint at SyS_write (fs/read_write.c:577) with arguments fd=1, buf=-1217257472, and count=18. Below the GDB window, a QEMU terminal window is visible, showing the MenuOS prompt and some system messages.

The image shows a dual-pane interface. The left pane displays a webpage titled "分析system_call中断处理过程". The article discusses how to trace a system call using GDB, specifically focusing on the transition from user space to kernel space via the `syscall` instruction. It mentions setting breakpoints at `SYSCALL_DEFINE3` and `fdput_pos`. A sidebar on the right contains navigation links like "切换界面", "关闭环境", "剪切板", etc.

The right pane shows a terminal window running GDB. The prompt is `(gdb) c`, indicating the program has continued execution. The output shows a breakpoint hit at `Sys_write` (line 577), followed by arguments being passed to `fs/read_write.c:577`. The next command shown is `(gdb) bt`, which triggers a backtrace, resulting in the message "Cannot access memory at address 0xfbad2a88".

断点命中证明内核的 `write` 处理函数正在执行。截图中的调用栈包含 `write`，随后出现无法读取内存的提示，说明 GDB 没能从内核汇编入口继续正确展开到用户态调用者。这不代表断点失败；本实验将 GDB 命中和参数作为动态证据，将 `system_call` 到返回路径的执行关系与 `entry_32.S` 源码对照分析，不把源码阅读描述成 GDB 单步结果。

调试过程中，断点也曾在 initramfs 解包阶段命中内核内部的 xwrite 路径。那条路径不是用户态程序通过 int 0x80 进入系统调用的证据。筛选证据时，结合调用栈、fd 和 MenuOS 操作上下文，采用 fd=1 的命中作为本次报告的主要动态证据。

三、system_call 入口与系统调用分派

在 32 位 x86 Linux 中，用户程序执行 `int 0x80` 后，CPU 根据中断向量 `0x80` 转入内核设置的入口。Linux 3.18.6 在初始化时将该向量关联到 `system_call`。发生从用户态到内核态的特权级转换时，CPU 保存返回所需的用户态状态；入口汇编随后保存通用寄存器等现场。

The screenshot displays a Linux terminal environment with a dark theme. The main window shows a C program named `system_call.c` that uses `gdb` to trace system calls. The code includes headers like `unistd.h`, `sys/types.h`, `fcntl.h`, `stdio.h`, `stdlib.h`, `string.h`, `sys/time.h`, `sys/wait.h`, `sys/resource.h`, `sys/mman.h`, `sys/stat.h`, `sys/uio.h`, `sys/xattr.h`, `sys/ipc.h`, `sys/shm.h`, `sys/msg.h`, `sys/socket.h`, `sys/un.h`, `sys/eventfd.h`, `sys/signalfd.h`, `sys/timerfd.h`, `sys/prctl.h`, `sys/poll.h`, `sys/select.h`, `sys/syscall.h`, `sys/sysinfo.h`, `sys/syslog.h`, `sys/time.h`, `sys/times.h`, `sys/timex.h`, `sys/uio.h`, `sys/xattr.h`, `sys/yield.h`, `sys/zfs_ioctl.h`, `sys/zfs_zerofree.h`, `sys/zfs_zonemap.h`, `sys/zfs_zonemap_ioctl.h`, `sys/zfs_zonemap_ioctl2.h`, `sys/zfs_zonemap_ioctl3.h`, `sys/zfs_zonemap_ioctl4.h`, `sys/zfs_zonemap_ioctl5.h`, `sys/zfs_zonemap_ioctl6.h`, `sys/zfs_zonemap_ioctl7.h`, `sys/zfs_zonemap_ioctl8.h`, `sys/zfs_zonemap_ioctl9.h`, `sys/zfs_zonemap_ioctl10.h`, `sys/zfs_zonemap_ioctl11.h`, `sys/zfs_zonemap_ioctl12.h`, `sys/zfs_zonemap_ioctl13.h`, `sys/zfs_zonemap_ioctl14.h`, `sys/zfs_zonemap_ioctl15.h`, `sys/zfs_zonemap_ioctl16.h`, `sys/zfs_zonemap_ioctl17.h`, `sys/zfs_zonemap_ioctl18.h`, `sys/zfs_zonemap_ioctl19.h`, `sys/zfs_zonemap_ioctl20.h`, `sys/zfs_zonemap_ioctl21.h`, `sys/zfs_zonemap_ioctl22.h`, `sys/zfs_zonemap_ioctl23.h`, `sys/zfs_zonemap_ioctl24.h`, `sys/zfs_zonemap_ioctl25.h`, `sys/zfs_zonemap_ioctl26.h`, `sys/zfs_zonemap_ioctl27.h`, `sys/zfs_zonemap_ioctl28.h`, `sys/zfs_zonemap_ioctl29.h`, `sys/zfs_zonemap_ioctl30.h`, `sys/zfs_zonemap_ioctl31.h`, `sys/zfs_zonemap_ioctl32.h`, `sys/zfs_zonemap_ioctl33.h`, `sys/zfs_zonemap_ioctl34.h`, `sys/zfs_zonemap_ioctl35.h`, `sys/zfs_zonemap_ioctl36.h`, `sys/zfs_zonemap_ioctl37.h`, `sys/zfs_zonemap_ioctl38.h`, `sys/zfs_zonemap_ioctl39.h`, `sys/zfs_zonemap_ioctl40.h`, `sys/zfs_zonemap_ioctl41.h`, `sys/zfs_zonemap_ioctl42.h`, `sys/zfs_zonemap_ioctl43.h`, `sys/zfs_zonemap_ioctl44.h`, `sys/zfs_zonemap_ioctl45.h`, `sys/zfs_zonemap_ioctl46.h`, `sys/zfs_zonemap_ioctl47.h`, `sys/zfs_zonemap_ioctl48.h`, `sys/zfs_zonemap_ioctl49.h`, `sys/zfs_zonemap_ioctl50.h`, `sys/zfs_zonemap_ioctl51.h`, `sys/zfs_zonemap_ioctl52.h`, `sys/zfs_zonemap_ioctl53.h`, `sys/zfs_zonemap_ioctl54.h`, `sys/zfs_zonemap_ioctl55.h`, `sys/zfs_zonemap_ioctl56.h`, `sys/zfs_zonemap_ioctl57.h`, `sys/zfs_zonemap_ioctl58.h`, `sys/zfs_zonemap_ioctl59.h`, `sys/zfs_zonemap_ioctl60.h`, `sys/zfs_zonemap_ioctl61.h`, `sys/zfs_zonemap_ioctl62.h`, `sys/zfs_zonemap_ioctl63.h`, `sys/zfs_zonemap_ioctl64.h`, `sys/zfs_zonemap_ioctl65.h`, `sys/zfs_zonemap_ioctl66.h`, `sys/zfs_zonemap_ioctl67.h`, `sys/zfs_zonemap_ioctl68.h`, `sys/zfs_zonemap_ioctl69.h`, `sys/zfs_zonemap_ioctl70.h`, `sys/zfs_zonemap_ioctl71.h`, `sys/zfs_zonemap_ioctl72.h`, `sys/zfs_zonemap_ioctl73.h`, `sys/zfs_zonemap_ioctl74.h`, `sys/zfs_zonemap_ioctl75.h`, `sys/zfs_zonemap_ioctl76.h`, `sys/zfs_zonemap_ioctl77.h`, `sys/zfs_zonemap_ioctl78.h`, `sys/zfs_zonemap_ioctl79.h`, `sys/zfs_zonemap_ioctl80.h`, `sys/zfs_zonemap_ioctl81.h`, `sys/zfs_zonemap_ioctl82.h`, `sys/zfs_zonemap_ioctl83.h`, `sys/zfs_zonemap_ioctl84.h`, `sys/zfs_zonemap_ioctl85.h`, `sys/zfs_zonemap_ioctl86.h`, `sys/zfs_zonemap_ioctl87.h`, `sys/zfs_zonemap_ioctl88.h`, `sys/zfs_zonemap_ioctl89.h`, `sys/zfs_zonemap_ioctl90.h`, `sys/zfs_zonemap_ioctl91.h`, `sys/zfs_zonemap_ioctl92.h`, `sys/zfs_zonemap_ioctl93.h`, `sys/zfs_zonemap_ioctl94.h`, `sys/zfs_zonemap_ioctl95.h`, `sys/zfs_zonemap_ioctl96.h`, `sys/zfs_zonemap_ioctl97.h`, `sys/zfs_zonemap_ioctl98.h`, `sys/zfs_zonemap_ioctl99.h`.

The sidebar on the right contains several icons and labels:

- 🏠 分析system_call中断处理过程
- 📁 系统调用列表参见 [torvalds/linux](#), 推荐在实验楼 Linux 虚拟机环境下完成实验。
- 📄 根据本周所学知识分析系统调用的过程。从 system_call 开始到 iret 结束之间的整个过程，并画出简要准确的流程图，撰写一篇署名博客，并在博客文章中注明“真实姓名（与最后申请证书的姓名务必一致）+ 原创作品转载请注明出处 + 《Linux 内核分析》MOOC 课程 [http://mooc.study.163.com/course/USTC-1000029000](#)”，博客内容的具体要求如下：
 - 题目自拟，内容围绕系统调用 system_call 的处理过程进行；
 - 博客内容中需要仔细分析 system_call 对应的汇编代码的工作过程，特别注意系统调用返回 iret 之前的进程调度时机等。
 - 总结部分需要阐明自己对“系统调用处理过程”的理解，进一步推广到一般的中断处理过程。
- 💡 请提交博客文章 URL 到网易云课堂 MOOC 平台，编辑成一个链接可以直接点击打开。（学员在线上学的学员注意提交博客 URL 到学堂在线上）
- 🔍 应用程序菜单
- 👤 shivanlou@6ab7a...
- 🐳 qemu-kernel linu...
- 🔧 gdb

截图中的关键入口逻辑可以按以下顺序理解：

1. system_call 入口保存原始 eax，并通过 SAVE_ALL 保存通用寄存器现场；GET_THREAD_INFO 获取当前线程信息。
2. 检查是否需要系统调用入口跟踪等额外处理，并验证 eax 中的系统调用号是否在有效范围内。
3. 执行 call *sys_call_table(%eax,4)，按系统调用号索引系统调用表。32 位表项宽度为 4 字节；write 的系统调用号为 4，因此分派到 write 对应的内核处理函数。
4. 系统调用处理函数完成后，返回值在 eax 中；入口代码将它保存到 pt_regs 对应位置，供恢复用户现场时带回。

本实验在 SyS_write 处观察参数 fd=1 和 count=18。它与上周 write 实验中“文件描述符、缓冲区、字节数”这三参数含义一致。入口截图展示的是 32 位系统调用表的通用分派代码；GDB 截图展示的是该分派最终到达 write 内核处理函数后的状态。

四、退出路径与返回前的调度时机

系统调用处理函数返回后，system_call 的退出路径会检查当前线程是否有待处理工作。若没有需要处理的标志，就进入 restore_all，恢复之前保存的寄存器现场，再执行中断返回。若有待处理工作，则转入 syscall_exit_work 等退出处理路径，进一步检查信号、跟踪请求和重新调度标志。

分析 system_call 中断处理过程

- 使用 gdb 跟踪分析一个系统调用内核函数（您上周选择的那个系统调用），系统调用列表参见 [torvalds/linux](#)，推荐在实验楼 Linux 虚拟机环境下完成实验。
- 根据本周所学知识分析系统调用的过程，从 system_call 开始到 iret 结束之间的整个过程，并画出简要准确的流程图，撰写一篇署名博客，并在博客文章中注明“真实姓名（与最后申请证书的姓名务必一致）+ 原创作品转载请注明出处 + 《Linux 内核分析》MOOC 课程 <http://mooc.study.163.com/course/USTC-1000029000>”，博客内容的具体要求如下：
 - 题目自拟，内容围绕系统调用 system_call 的处理过程进行；
 - 博客内容中需要仔细分析 system_call 对应的汇编代码的工作过程，特别注意系统调用返回 iret 之前的进程调度时机等。
 - 总结部分需要阐明自己对“系统调用处理过程”的理解，进一步推广到一般的中断处理过程。

请提交博客文章 URL 到网易云课堂 MOOC 平台，编辑成一个链接可以直接点击打开。（学堂在线上的学员注意提交博客 URL 到学堂在线上）

完成学习

30:05 定时

切换界面

关闭环境

剪贴板

共享桌面

下载代码

上传文件

桌面全屏

环境信息

SSH 连接

保存环境

更多会员权益

新手指南

work_pending 会检查当前任务的 need_resched 标志。需要重新调度时，执行 call schedule；处理完后再回到用户态返回准备路径。若有待处理信号，则进入相应的信号处理路径。最后，内核恢复寄存器现场，并通过 INTERRUPT_RETURN 返回。此 32 位入口中的 INTERRUPT_RETURN 对应 iret 返回指令。

简化流程如下：

用户态发起 32 位 write 系统调用（系统调用号 4，参数按 ABI 传递）

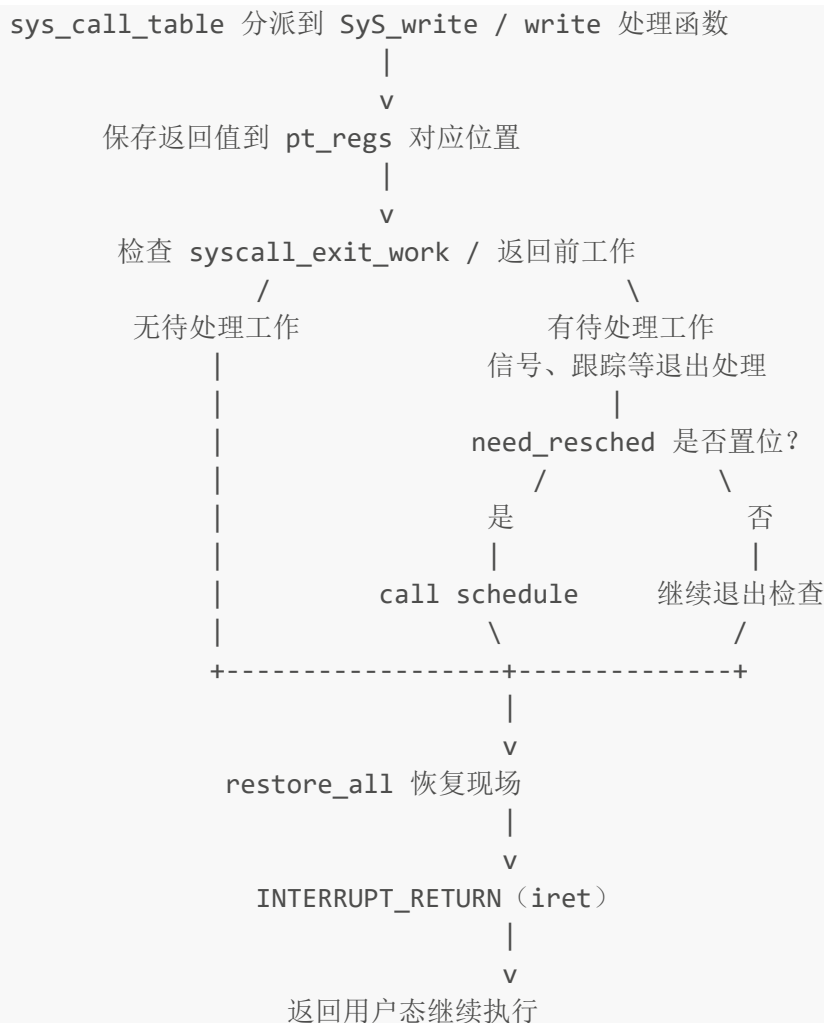
↓

CPU 特权级转换并保存返回所需的状态

↓

system_call: 保存现场、检查入口工作与系统调用号

↓



这里的 `schedule` 是**条件性的返回前调度时机**，不是每次 `write` 都会发生进程切换。只有退出路径检查到需要重新调度时才会调用。发生切换后，原任务的内核栈和现场仍保留；等它以后再次获得 CPU，才继续自己的退出路径并最终返回用户态。

CPU 在 `int 0x80` 导致特权级转换时保存的返回状态，与汇编入口通过 `SAVE_ALL` 保存的通用寄存器现场，是相互配合的两部分。返回时，内核先恢复软件保存的寄存器，再由 `iret` 恢复中断帧中的控制状态，从而回到用户态的后续指令。

五、对系统调用处理过程的理解

这次跟踪让我把用户态接口、系统调用入口、内核服务分派和中断返回连成一条路径：用户程序按 ABI 准备系统调用号和参数，CPU 经由中断门切换到内核态，`system_call` 保存现场并查表调用内核服务，服务结果进入返回现场，内核在 `iret` 之前检查信号和调度等工作，最后恢复现场回到用户态。

GDB 在 C 处理函数处提供了参数和源码位置的动态证据；汇编入口和返回代码则通过对应版本的 `entry_32.S` 分析。GDB 无法展开跨越汇编入口的完整调用栈时，应明确证据边界，再用源码解释控制流。普通中断处理也遵循相似的基本结构：保存现场、执行处理程序、检查必要的后续工作、恢复现场并返回；系统调用的特别之处在于它由用户程序主动触发，并按系统调用号选择内核服务。

参考资料

- Linux v3.18.6 源码：arch/x86/kernel/entry_32.S、arch/x86/syscalls/syscall_32.tbl、fs/read_write.c。系统调用号可查 [32 位 x86 系统调用表](#)。

- 孟宁等：《庖丁解牛 Linux 内核分析》第 5 章 5.3 节“系统调用在内核代码中的处理过程”及第 8 章 8.1.2 节“进程调度时机”。
- [系统调用分析参考网页](#)
- [《Linux 内核分析》MOOC 课程](#)

署名与转载说明

叶丁再（学号：20262809，姓名与证书申请信息一致）。

原创作品转载请注明出处：《Linux 内核分析》MOOC 课程：

<http://mooc.study.163.com/course/USTC-1000029000>

博客发布 URL：发布后补充。本文目前为待发布 Markdown 文稿。